

Week 1

Let's start

Let's start with a simple floor plan DXF model. We simply want to extrude the walls to obtain a simple 3D model. Only at a later time will I also be able to consider the doors, windows and more. But for now let's keep it simple.

DXF does not give you polygons, it gives you disconnected entities:

1. POLYLINE: the most common entity for room outlines and walls.
2. LINE: individual segments, often used for walls too.
3. ARC/CIRCLE: doors. columns.
4. INSERT: furniture, door/window symbols.
5. LAYER: the key organizational tool.

The main challenge with DXF files is that they are often bunch of independent of lines.

For each entity we only need a list of segments or ordered lists of vertices if the entity is already a polyline. Our internal storage could be something like this:

```
struct Segment{
    glm::dvec2 p1, p2;
};

struct Polyline {
    std::vector<glm::dvec2> points;
    bool closed
};
```

IMPORTANT: during the parsing we must filter the geometry. If we feed everything: furniture, dimension lines, hatching, text boundaries, annotation leaders into your triangulation pipeline, the results will be completely wrong. The triangulation algorithm needs a clean, simple polygon. So filtering is not optional, it's a prerequisite for correctness.

The parser class:

```
class DRWParser : public DRW_Interface
{
public:
    void addHeader(const DRW_Header* data);
    void addLine(const DRW_Line& data);
```

```

void addPolyline(const DRW_Polyline& data);
void addLWPolyline(const DRW_Polyline& data);

std::vector<Segment> segments;
std::vector<Polyline> polylines;
f32 unit_scale = 1.0f;
}

```

```

auto parser = DRWParser{};
auto dxf = dxfRW(file_path);
if (!dxf.read(&parser, false))
    return throw std::runtime_error();

```

```

> ./build/bin/Plan2Scene res/sample-floor-plan.dxf
Line: layer = WALLS
Line: layer = DOORS
Line: layer = WINDOWS
Successfully parsed DXF file. Segments: 5, Polylines: 0

> ./build/bin/Plan2Scene res/giraffe360_demo_commercial_1.dxf
LWPolyline: layer = WALLS nr_vertices = 30 closed = true

```

The parser should produce as output a list of contours, where each contour is already an ordered list of vertices. A LWPOLYLINE directly becomes one entry in the list, a collection of LINE segments goes through snap + chain and becomes one entry in the list.

In the case of LWPOLYLINE, the vertices are already ordered! We just extract them directly as your contour. Check whether the polyline is flagged as closed, if it's closed, the last vertex implicitly connects back to the first and you don't need to duplicate it.

Otherwise in the case of individual SEGMENTS, we have no ordering information. This is where the snap + chain algorithm is needed to reconstruct the contour.

CASE A: In the first case we have independent pairs of segments. They have no topological relationship to each other. The triangulation library needs an ordered sequence of vertices forming a closed polygon. So the goal of this phase is to convert unordered segments into ordered closed contour.

Before procede, we need to normalize the coordinates. Two segments that share an endpoint may have endpoints like (1.0000001, 2.0) and (0.9999999, 2.0) due to floating point. If you treat these as different points, the graph will have gaps and the chaining will fail. For every endpoint in every segment, round or snap it to a grid with tolerance ϵ : two points closer than ϵ become the same point.

Once points are snapped, we must build an adjacency graph: think of each unique point as a graph node, and each segment as an undirected edge between two nodes. For a valid simple room contour, every node should have exactly degree 2. Starting from any node, follow edges greedily: pick a neighbor you haven't visited, move there, repeat until you return to the start. This gives you your ordered vertex sequence.

CASE B: In the second case, we already have a polyline with 30 ordered vertices, we already have exactly what you need, i.e. an ordered contour.

If the geometry is already a LWPOLYLINE, the parser gives you the vertices in order. The chaining algorithm exists to handle the case where your geometry arrives as unordered disconnected segments. If polyline is flagged `closed = true`, DXF already tells the polyline is closed. However, due to floating-point, `v[last]`, `v[0]` might not be bitwise identical. For this reason we should check the distance between them: if their distance is less than ϵ they represent the same logical point. We can drop the last vertex so the contour doesn't have a near-duplicate.

Otherwise, if polyline is flagged `closed = false` we can check the distance between `v[0]`, `v[last]`, if it's less than ϵ the polyline is geometrically closed. We merge by popping the last vertex and treat it as a closed contour. Otherwise if the distance is greater, the polyline is genuinely open; for wall extrusion this is problematic because we need a closed polygon.

```
// Is polyline closed: we should check the distance between them v[0] and v[last] and if their  
// distance is less than epsilon they represent the same logical point.  
// We can drop the last vertex so the contour doesn't have a near-duplicate.  
if(wall_polyline.closed)  
{  
    auto first_point = wall_points.front();  
    auto last_point = wall_points.back();  
    if(distance(first_point, last_point) < epsilon)  
        wall_points.pop_back();  
}
```

```
// Polyline is open: we should check if the first and last point are close enough to be  
// considered the same point.  
else  
{  
    auto first_point = wall_points.front();  
    auto last_point = wall_points.back();  
    if(distance(first_point, last_point) < epsilon)  
    {  
        wall_points.pop_back();  
        wall_polyline.closed = true;  
    }  
    else  
        throw std::runtime_error();  
}
```

Now we have one clean contour: an ordered list of 2D vertices, guaranteed closed, with no duplicate endpoints. This is our floor polygon! The next concrete step is integrating `poly2tri`.

Poly2tri implements Constrained Delaunay Triangulation. It takes a closed polygon and fills its interior

with triangles.

This API has some important constraints: no duplicate points, no self-intersecting contours.

Poly2tri expects the outer polygon to be counter-clockwise (CCW) and holes to be clockwise (CW). We compute signed area with shoelace formula to determine orientation.

```
auto area = signed_area(wall_polyline);
if(area < 0)
    std::ranges::reverse(wall_points);

auto contour = std::vector<p2t::Point*>{};
for(const auto& p : wall_points)
    contour.push_back(new p2t::Point{p.x, p.y});

auto cdt = p2t::CDT(contour);
cdt.Triangulate();
auto triangle_list = cdt.GetTriangles();
```

Now we have a vector of triangles where each triangle gives us 3 points. This is our floor mesh.

Now let's define the layout of a vertex as 3 floats + 3 floats:

```
struct Vertex {
    glm::vec3 position;
    glm::vec3 normal;
};
```

To see if everything went well, we can try to visualize the floor: we have the triangles and therefore the vertices, we can already create a simple mesh and with OpenGL and GLSL we can render the mesh.

```
void main()
{
    vec3 n = normalize(vs_out_normal_world_space);
    n = n * 0.5 + 0.5;
    fs_out_color = vec4(n, 1.0);
}
```

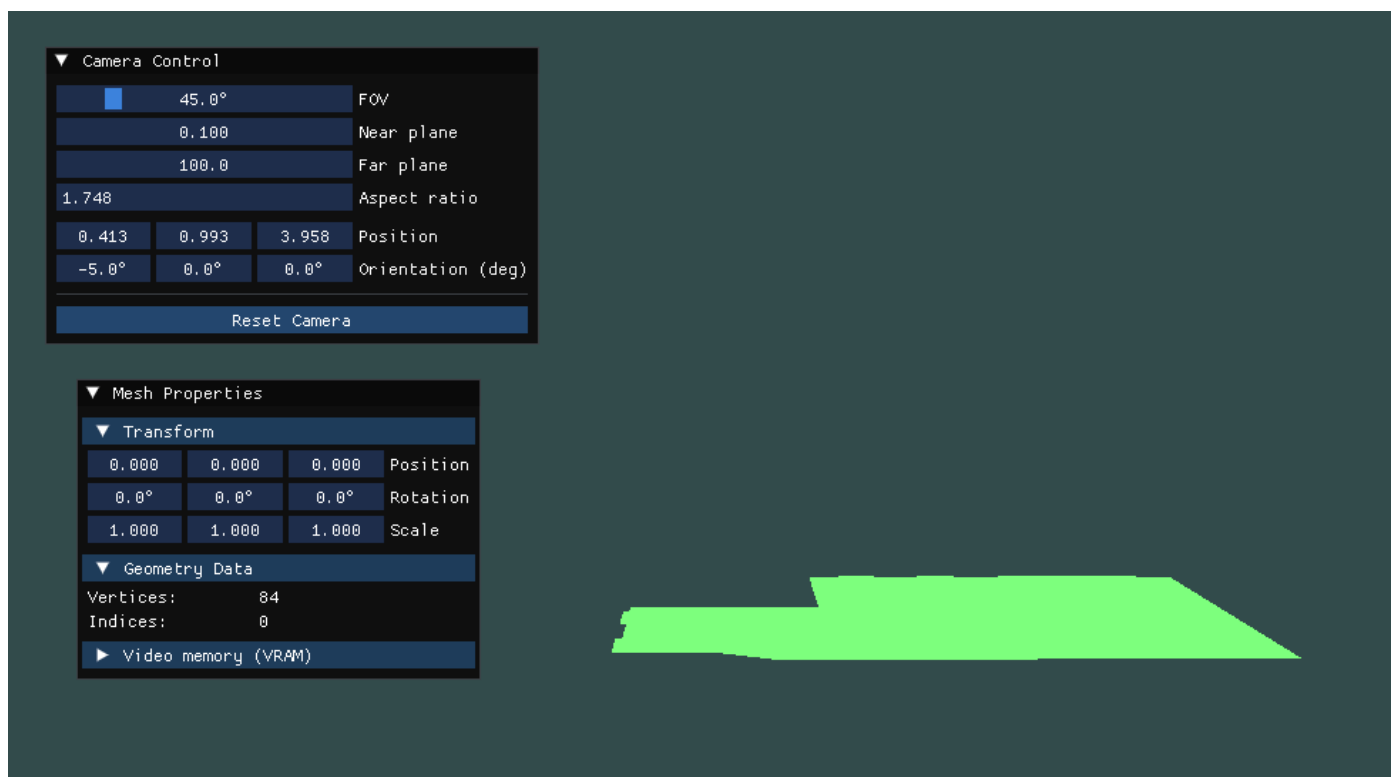
```
auto vertices = std::vector<Vertex>{};
for (const auto& tri : triangle_list)
{
    for (auto i = 0; i < 3; ++i)
    {
        auto p = tri->GetPoint(i);
        auto v = Vertex{};
        v.position.x = f32(p->x);
        v.position.y = 0.f; // floor at y=0
        v.position.z = f32(p->y);
    }
}
```

```

        v.normal = {0.f, 1.f, 0.f}; // normal points up (+Y)
        vertices.push_back(v);
    }
}

auto visualizer = MeshVisualizer(1024, 768);
visualizer.set_mesh(std::make_shared<StaticMesh>(
    vertices.data(),
    vertices.size(),
    nullptr,      // indices
    0             // num indices ok bene
));
visualizer.render();

```



The next step is wall extrusion. For each edge of the contour ($v[i]$, $v[i+1]$), create a vertical quad. Each quad is two triangles: (BL, BR, TR) and (BL, TR, TL), where:

- $BL = (v[i].x, v[i].y, 0)$
- $BR = (v[i+1].x, v[i+1].y, 0)$
- $TR = (v[i+1].x, v[i+1].y, H)$
- $TL = (v[i].x, v[i].y, H)$

For architectural drawings a standard ceiling is 2.7-3.0 meters.

```

for (auto i = 0u; i < wall_points.size(); ++i)
{
    auto p1 = wall_points.at(i);

```

```

auto p2 = wall_points.at((i + 1) % wall_points.size());

// outward normal: edge direction in XZ plane rotated 90 degrees
auto dx = f32(p2.x - p1.x);
auto dz = f32(p2.y - p1.y);
auto len = std::sqrt(dx * dx + dz * dz);
auto normal = glm::vec3{dz / len, 0.f, -dx / len};

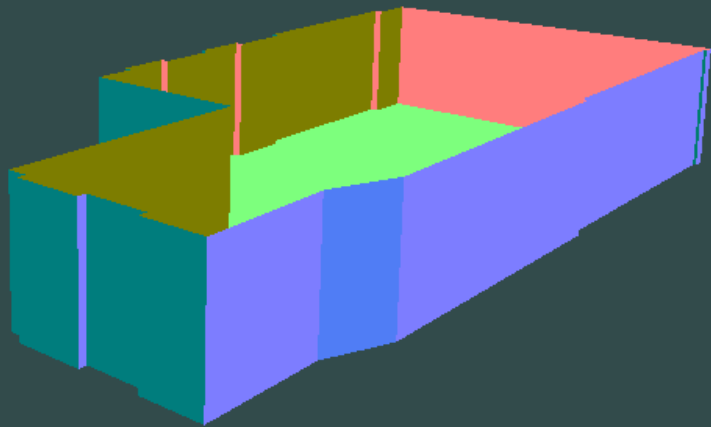
// 4 corners of the wall quad, Y-up convention
auto BL = Vertex{ .position={f32(p1.x), 0.f, f32(p1.y)},
.normal=normal};
auto BR = Vertex{ .position={f32(p2.x), 0.f, f32(p2.y)},
.normal=normal};
auto TR = Vertex{ .position={f32(p2.x), H, f32(p2.y)},
.normal=normal};
auto TL = Vertex{ .position={f32(p1.x), H, f32(p1.y)},
.normal=normal};

// triangle 1: BL, BR, TR
vertices.push_back(BL);
vertices.push_back(BR);
vertices.push_back(TR);
// triangle 2: BL, TR, TL
vertices.push_back(BL);
vertices.push_back(TR);
vertices.push_back(TL);
}

```

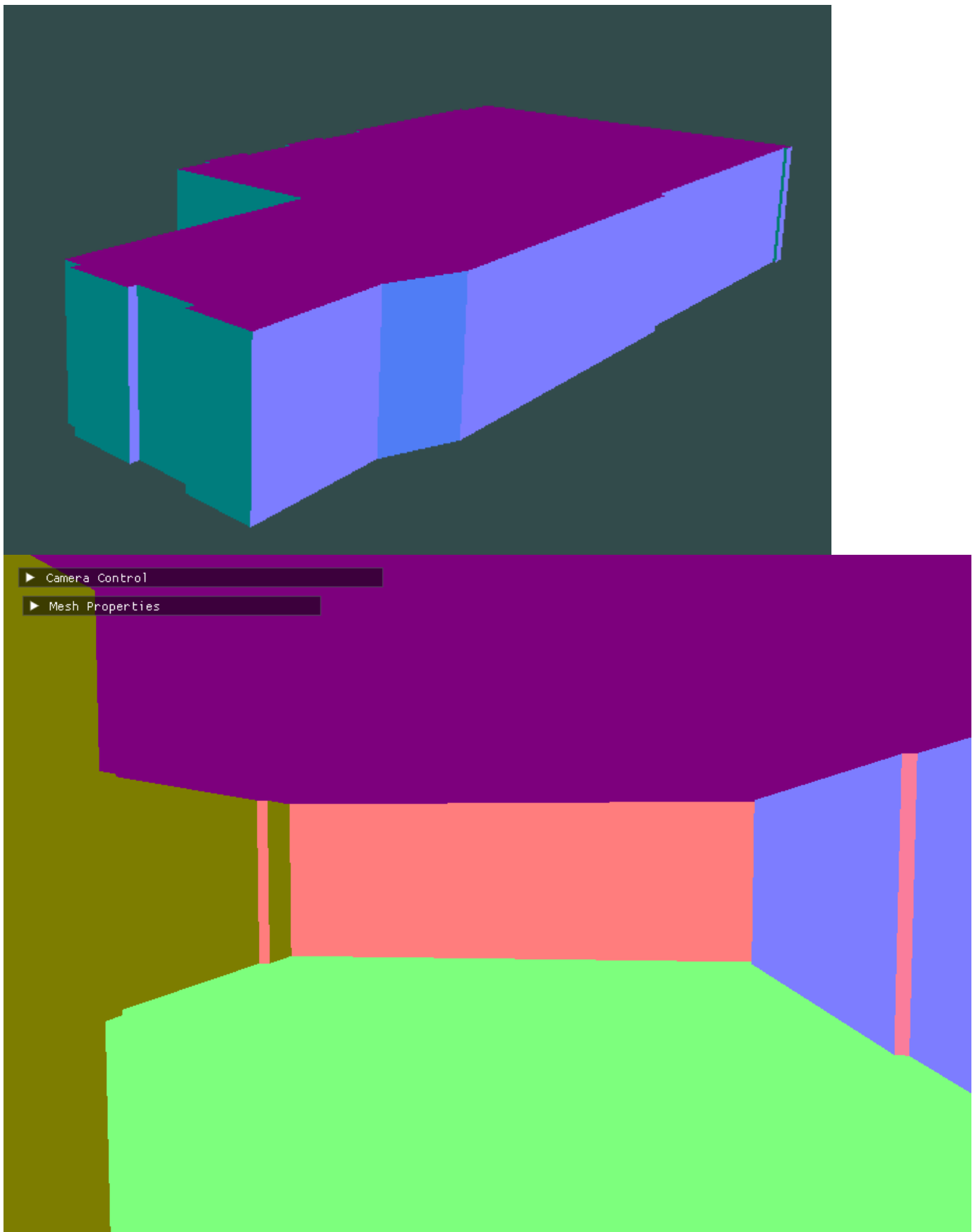
▼ Camera Control			
<input type="range"/>	45.0°	FOV	
	0.100	Near plane	
	100.0	Far plane	
	1.404	Aspect ratio	
-2.502	1.888	4.164	Position
-11.0°	-46.0°	-0.0°	Orientation (deg)
Reset Camera			

▼ Mesh Properties			
▼ Transform			
0.000	0.000	0.000	Position
0.0°	0.0°	0.0°	Rotation
0.002	0.002	0.002	Scale
▼ Geometry Data			
Vertices:	264		
Indices:	0		
▶ Video memory (VRAM)			



Finally, the ceiling is same triangles as floor but at height H and normal pointing down:

```
for (const auto& tri : triangle_list)
{
    for (auto i = 0; i < 3; ++i)
    {
        auto p = tri->GetPoint(i);
        auto v = Vertex{};
        v.position.x = f32(p->x);
        v.position.y = H;
        v.position.z = f32(p->y);
        v.normal = {0.f, -1.f, 0.f};
        vertices.push_back(v);
    }
}
```



One of the main problems encountered is that of reading the unit of measurement. During the parsing of the DXF model, first of all we must read the metadata (header) to understand mainly which unit of measurement has been adapted for the creation of the model:

```
void DRWParser::addHeader(const DRW_Header* data)
{
```



```

auto it = data->vars.find("$INSUNITS");
auto variant = it->second;
auto units = variant->content.i;
switch (units)
{
    case 1:
        unit_scale = 0.0254f; // inches
        break;
    case 4:
        unit_scale = 0.001f; // millimeters
        break;
    case 5:
        unit_scale = 0.01f; // centimeters
        break;
    case 6:
        unit_scale = 1.0f; // meters
        break;
    default:
        unit_scale = 0.0f; // unknown
        break;
}
}

```

If we don't know which unit of measurement has been adopted, the solution is to calculate it based on the geometry, extracting the points and calculating the bounding box:

```

auto detect_unit_scale(const std::vector<glm::dvec2>& points) -> f32
{
    auto min_x = std::numeric_limits<double>::max();
    auto max_x = std::numeric_limits<double>::lowest();
    auto min_y = std::numeric_limits<double>::max();
    auto max_y = std::numeric_limits<double>::lowest();
    for (const auto& p : points)
    {
        min_x = std::min(min_x, p.x); max_x = std::max(max_x, p.x);
        min_y = std::min(min_y, p.y); max_y = std::max(max_y, p.y);
    }
    const auto extent = std::sqrt(
        std::pow(max_x - min_x, 2) +
        std::pow(max_y - min_y, 2));

    if (extent > 5000.0) return 0.001f; // mm
    else if (extent > 500.0) return 0.01f; // cm
    else if (extent > 50.0) return 0.1f; // dm
}

```

```
    else return 1.0f;    // meters
}
```

This function simply returns a multiplier that you then apply manually to each point. It just figures out what unit scale should be. The actual conversion happens in the for loop:

```
if(parser.unit_scale == 0.0f)
    parser.unit_scale = detect_unit_scale(wall_points);

for (auto& p : wall_points)
{
    p.x *= parser.unit_scale;
    p.y *= parser.unit_scale;
}
```